

DW Wellness AI — Application Architecture

A handoff document describing the architecture of **Dimensional Wellness AI (DWAI)**, a consent-based personal wellness assistant. This is written so another AI (or engineer) can understand the system end-to-end and suggest improvements or restructuring. Last compiled: June 2026.

1. What the App Is

Dimensional Wellness AI is a full-stack web + mobile application that acts as an adaptive, energy-based personal wellness concierge. Instead of prescriptive routines, it follows a "**Pause → Name → Flip → Choose**" philosophy with an emphasis on explicit user consent, energy awareness, and a no-guilt, non-medical approach.

Core surfaces:

- **Command Center** — main dashboard with a central "DW Orb" AI interaction point.
- **Talk** — conversational AI chat with deep app integration (the AI can read context and take actions).
- **Calendar** — Apple-Calendar-style day/week/month with AI lifestyle suggestions.
- **Smart Import** — universal content parser (PDFs, chats, docs, images) that maps content into the user's "life system."
- **Domain pages** — Goals, Habits, Workout, Finances, Spiritual, Relationships, Journal, Browse, Cosmic Hub, and more.

The product is delivered as a **single React SPA** served by an **Express API**, packaged for **iOS/Android via Capacitor** (same web build wrapped in a native shell, plus native voice/Siri/Google Assistant integrations).

2. High-Level Architecture



3. Technology Stack

Frontend

- **React 18.3 + TypeScript**
- **Vite** — build tool / dev server
- **Wouter 3** — lightweight client-side routing
- **TanStack React Query 5** — server-state caching/fetching (default fetcher configured globally)
- **Radix UI + shadcn/ui** — component primitives (`components.json`)
- **Tailwind CSS** — styling, dark-mode via `class` strategy
- **React Hook Form + Zod** — forms and validation
- **Recharts** — data visualization
- **Space Grotesk** (display) + **Nunito** (body) typography

Backend

- **Node.js + Express 4 in TypeScript (ESM)**
- **Drizzle ORM 0.45 + drizzle-zod + Drizzle Kit** (migrations)

- **express-session** + **connect-pg-simple** (Postgres-backed sessions)
- **Passport** (Google + Facebook OAuth strategies); email/password handled directly
- **ws** — WebSocket server for realtime voice/chat
- **Zod** — request validation (insert schemas derived from Drizzle tables)

Data

- **PostgreSQL** (Neon serverless driver, `@neondatabase/serverless`)
- **~140 tables**, single shared schema in `shared/schema.ts`
- **33 SQL migration files** in `migrations/`

Mobile

- **Capacitor 8** — wraps the web build for iOS/Android (`capacitor.config.ts` , `ios/` , `android/`)
- Native voice assistant: Siri/App Intents (iOS), Google Assistant App Actions (Android)

Build / tooling

- `npm run dev` → `tsx watch server/index.ts` (Express serves the Vite dev middleware on one port, **port 5000**)
- `npm run build` → custom `script/build.ts` (bundles client + server to `dist/`)
- `npm run start` → `node dist/index.cjs` (production)
- **Vitest** + **Playwright** for unit/e2e tests
- Prettier + Husky (formatting / git hooks)

4. Repository Layout

client/	React SPA
src/	
App.tsx	route table (Wouter), providers, lazy loading
main.tsx	entry
pages/	~97 page components (one per route)
components/	~111 shared components (incl. shadcn ui/)
contexts/	React context providers (theme, auth, etc.)
hooks/	custom hooks (use-toast, usePageMeta, etc.)
lib/	client utilities (queryClient, billing, etc.)
stores/	client state stores
copy/	centralized UI copy/messaging layer
config/, core/, features/, routes/	
server/	Express API
index.ts	bootstrap: middleware, route-audit, schedulers
routes.ts	session/auth/OAuth setup + central route wiring
routes/	~65 feature route modules (see §6)
storage.ts	IStorage interface – all DB access goes here
lib/	user-context (AI), route-audit, today-brief, etc.
db.ts	Drizzle client / pool
openai.ts	OpenAI + Perplexity calls, prompt assembly
ai-engine.ts	higher-level AI orchestration
migrate.ts	production migration runner (boot-time)
dev-db-bootstrap.ts	dev DB self-healing migration replay
middleware/	Express middleware
personality/	AI persona/presets
email.ts, push.ts, plaid-sync.ts, proactive.ts, *-scheduler.ts	
shared/	
schema.ts	Drizzle tables + Zod insert schemas + types (single source of truth for client + server)
migrations/	33 SQL migration files (drizzle-kit)
scripts/, script/	build, seeding, schema-drift checks
ios/, android/	Capacitor native projects
artifacts/	mockup sandbox (dev-only, NOT deployed)

5. Data Model (PostgreSQL via Drizzle)

`shared/schema.ts` defines ~**140 tables** as the single source of truth. For each table it exports the Drizzle table, a `createInsertSchema` Zod schema, an insert type, and a select type — used identically on client and server.

Major domains (representative tables):

- **Identity & account:** `users`, `user_profiles`, `onboarding_profiles`, `password_reset_tokens`, `usage_meters`, `feature_settings`
- **Life system / planning:** `life_systems`, `projects`, `project_chats`, `project_milestones`, `project_artifacts`, `universal_plans`, `dimension_blueprints`, `dimension_systems`, `system_modules`, `user_system_preferences`
- **Goals / habits / routines:** `goals`, `habits`, `habit_logs`, `routines`, `routine_logs`, `tasks`, `streaks`, `achievements`, `completion_status`
- **Calendar / scheduling:** `calendar_events`, `calendar_event_tasks`, `schedule_blocks`, `daily_schedule_events`, `reminders`, `reminder_ledger`
- **AI / conversations:** `conversations`, `ai_learnings`, `ai_sync_sessions`, `ai_sync_items`, `ai_pattern_snapshots`, `ai_suggestions`, `ai_feature_usage`, `conversation_insights`, `dw_insights`, `dw_followups`,

`dw_journal_entries`, `interaction_events`

- **Wellness tracking:** `mood_logs`, `daily_mood_checks`, `mood_insights`, `check_ins`, `evening_check_ins`, `activity_completions`, `tracking_logs`, `water_logs`, `meal_logs`, `health_metrics`, `body_scans`
- **Stress / recovery:** `wellness_blueprints`, `baseline_profiles`, `stress_signals`, `stabilizing_actions`, `support_preferences`, `recovery_reflections`, `reset_protocol`, `user_patterns`
- **Nutrition / fitness:** `meal_plans`, `meals`, `meal_prep_preferences`, `shopping_lists`, `shopping_list_items`, `workout_plans`, `exercises`, `workout_sessions`, `workout_session_steps`
- **Cosmic hub:** `birth_charts`, `astrology_predictions`
- **Relationships (Social Environment):** `people`, `people_interactions`, `aliveness_moments`, `relationship_boundaries`, `relationship_repairs`, `relationship_appreciations`, `people_groups`, `people_group_members`, `group_shared_items`, `relationship_insights`
- **Finances:** Plaid-linked items/accounts/transactions/holdings + savings goals (finance routes)
- **Wearables:** `wearable_devices`, `wearable_data`
- **Imports:** `imported_documents`, `imported_document_items`, plus `imported_conversations` (ChatGPT import)
- **Notifications / scheduling infra:** `notifications`, `notification_preferences`, `push_subscriptions`, `vapid_keys`, `scheduler_leases`, `monitoring_alerts`
- **Accountability:** `accountability_partners`, `task_accountability`, `accountability_stats`
- **Content / browse:** `wellness_content`, `saved_content`, `feed_interactions`, `challenges`
- **Daily brief:** `daily_briefs`, `daily_brief_taps`, `daily_brief_preferences`

Migration strategy

- **Production:** `server/migrate.ts` (`runMigrations`) replays `migrations/*.sql` via Drizzle Kit's migrator on boot.
- **Dev:** `server/dev-db-bootstrap.ts` (`bootstrapDevDb`) replays the same SQL but tolerates "already exists/gone" errors so the dev DB self-heals on every `npm run dev`. It also drops a fixed set of orphan tables. This is the **only** dev migration entry point (no `drizzle-kit push` in the post-merge flow).
- **Drift enforcement:** a CI `schema-drift` job boots a clean Postgres, runs the bootstrap, and asserts every `pgTable(...)` column exists (and no unexpected orphan tables remain).

6. Backend Architecture

Request lifecycle

1. `server/index.ts` boots Express, installs middleware, a **route-audit** wrapper (warns on duplicate method+path registrations), then registers routes and starts in-process schedulers.
2. `server/routes.ts` configures the **session** (`express-session` + `connect-pg-simple`), **Passport** OAuth (Google/Facebook), email/password auth, and wires in every feature module.
3. Feature route modules under `server/routes/` register their endpoints (each module owns a slice of the API).
4. Routes are kept thin: they validate input with **Zod**, then call the **IStorage** data layer (`server/storage.ts`) for all DB work.

Route modules (~65 files in `server/routes/`)

Grouped by concern:

- **Auth/account:** `auth-extra.ts`, `password-reset.ts`, `users.ts`, `onboarding.ts`, `household.ts`
- **Billing:** `billing.ts` (Stripe checkout/portal/webhook/status, gated on env vars)
- **AI/chat:** `ai-features.ts`, `chat-handlers.ts`, `conversations.ts`, `realtime.ts` (WebSocket voice/chat), `dw-process.ts`, `voice-extras.ts`, `music-explain.ts`
- **Planning/life-system:** `plans.ts`, `plans-shopping.ts`, `plan-builder` flows, `life-system-*.ts`, `elevation-*.ts`, `week-planner.ts`, `weekly-review.ts`, `tasks-projects.ts`, `system-modules.ts`, `dimensions-config.ts`
- **Calendar/today:** `calendar-routes.ts`, `today.ts`, `dashboard.ts`, `reminders.ts`

- **Domain features:** `goals-habits.ts`, `workout-suggest.ts`, `wellness-tracking.ts`, `health-metrics.ts`, `spiritual.ts`, `astrology.ts`, `finances.ts`, `plaid.ts`, `relationships.ts`
- **Content/browse:** `content-feed.ts`, `discover-feed.ts`, `browse-misc.ts`, `routines-browse.ts`, `local-resources.ts`, `media-misc.ts`
- **Imports:** `import-routes.ts`, `imports-chat.ts`, `documents.ts`, `life-system-import.ts`, `life-system-extract.ts`
- **Wearables:** `wearables.ts` (canonical — all `/api/wearables/*` live here), `wearable-providers.ts` (Whoop/Oura/Garmin OAuth)
- **Notifications/accountability:** `notifications.ts`, `accountability-routes.ts`, `checkins-blueprint.ts`, `checkin-status.ts`
- **Learning/insights:** `learning-profile.ts`, `learning-threads.ts`, `trigger-detection.ts`, `triggers.ts`, `weekly-review.ts`, `admin-progress.ts`, `analytics-health.ts`
- **Misc:** `feedback.ts`, `support-detailed.ts`, `profile-challenges.ts`, `helpers-routes.ts`

Architectural rule learned the hard way: Express runs only the FIRST handler for a given method+path, so duplicate registrations become silent dead code. `server/lib/route-audit.ts` logs warnings at startup; new wearable routes must only live in `wearables.ts`.

Storage layer

`server/storage.ts` exposes an `IStorage` interface implemented over Drizzle. All CRUD flows through it using `@shared/schema` types. This is the seam where user-scoping/ownership checks should live.

In-process schedulers

Several background jobs run inside the server process, coordinated by a DB **lease** (`scheduler_leases`) so only one instance runs each:

- Push reminder scheduler + health monitor
- Mood-insights scheduler
- Relationship-nudge scheduler
- Plaid sync (when configured)

7. AI Integration (the core of the product)

- **Models:** OpenAI `gpt-4o` (heavier reasoning/generation) and `gpt-4o-mini` (chat, faster, higher rate limits). Centralized in `server/openai.ts` and orchestrated by `server/ai-engine.ts`.
- **Content feed: Perplexity** (`sonar` model) powers Browse "For You"/articles/video personalization.
- **Context assembly:** `server/lib/user-context.ts` builds the per-user context (profile, goals, recent activity, **imported documents/excerpts**) injected into the AI's system prompt so replies are personalized and can reference the user's own material.
- **Tool calls / actions:** the Talk chat can take real actions in the app (create journal entries, set reminders, update goals, etc.) and fetch live context.
- **TTS:** all in-app speech uses OpenAI's **Alloy** voice through a unified `tts-service`, with personality presets and adjustable speed.
- **Spiritual/cosmic readings:** AI-generated daily personal readings, cached per user per day, with a deterministic template fallback on any AI failure.
- **Proactive intelligence:** the system proactively nudges based on energy levels, financial goals, habit reminders, and weekly summaries.

8. Authentication & Sessions

- **Cookie-based sessions** via `express-session`, persisted in Postgres (`connect-pg-simple`).
- **Email/password** auth plus **Google** and **Facebook** OAuth (Passport strategies); OAuth callbacks link to local accounts.
- **Password reset** uses high-entropy one-time tokens (`password_reset_tokens`) emailed via Resend (delivery requires a verified sending domain).
- **Guest mode**: unauthenticated users get LocalStorage-backed data on the client.
- Authenticated routes require a valid server-side session; data-bearing routes enforce per-user ownership (see threat model).

9. External Integrations

Provider	Purpose	Config (env)
OpenAI	Chat, generation, TTS (Alloy)	OpenAI credentials
Perplexity	Browse content feed (<code>sonar</code>)	Perplexity key
Stripe	Subscriptions (monthly/annual)	<code>STRIPE_SECRET_KEY</code> , <code>STRIPE_WEBHOOK_SECRET</code> , <code>STRIPE_PRICE_ID_MONTHLY</code> , <code>STRIPE_PRICE_ID_ANNUAL</code>
Plaid	Financial accounts/transactions	Plaid client/secret
Resend	Transactional email	Resend key + verified domain
Google Vision	OCR for imports	Vision credentials
Google/Facebook	OAuth login	OAuth client IDs/secrets
Whoop / Oura / Garmin	Wearable health data	per-provider client IDs/secrets
Replit Object Storage	Uploaded plan files	<code>PRIVATE_OBJECT_DIR</code> , <code>PUBLIC_OBJECT_SEARCH_PATHS</code>

Billing note: the Stripe integration is implemented with **plain env-var credentials** (real Checkout, customer portal, webhook sync) — only monthly/annual subscriptions, no one-time "lifetime" SKU. The client must use the real `/api/billing/checkout` flow and never grant entitlement locally.

10. Frontend Architecture

- **Routing**: Wouter; ~97 page components registered in `client/src/App.tsx`, lazy-loaded via React Suspense.
- **Server state**: TanStack Query with a global default fetcher; mutations use a shared `apiRequest` helper and invalidate by query key.
- **Design system**: shadcn/ui + Radix + Tailwind; global `app-shell` for safe-area handling; reusable `PageHeader`; `usePageMeta` for per-page SEO.
- **Theming**: dark mode via `class` strategy; cosmic background, time-of-day gradients, orbit rings in the Command Center.
- **Copy layer**: centralized UI copy for consistent tone.
- **Object uploads**: plan artifacts upload via multipart to the API, stored in Object Storage with text excerpts fed back into AI context.

11. Mobile (Capacitor)

- The web build is wrapped by **Capacitor 8** into native iOS/Android shells (`ios/` , `android/` , `capacitor.config.ts`).
 - Native voice assistant integration: **Siri / App Intents / App Shortcuts / widgets** (iOS) and **Google Assistant App Actions** (Android), plus an in-app voice conversation mode (STT/TTS) and deep-linking actions.
 - Build flow: `npm run build` → `npx cap sync` → open/build in Xcode/Gradle.
-

12. Security Posture (summary)

The app stores highly sensitive personal data (chats, moods, calendar, finances via Plaid, wearables, uploads). Key guarantees the architecture is designed to uphold:

- Authenticated routes require a valid server-side session.
 - Every update/delete-by-ID route enforces **per-user ownership**; batch endpoints verify each child belongs to the user.
 - Read endpoints scope data to the authenticated user before returning.
 - Secrets stay in environment storage; sensitive responses/tokens are never logged in production.
 - Public AI/auth endpoints have rate limits; upload/body sizes are bounded.
 - Object-level authorization (one user reading another's records) is the primary escalation risk and is guarded at the storage/route layer.
-

13. Known Architectural Notes & Tensions (good candidates for review)

These are honest weak points / tech-debt areas worth a second opinion:

1. **Route sprawl**: ~65 route modules plus a large `server/routes.ts` . Some endpoints have historically been registered in multiple places (mitigated by a route-audit warner, but the underlying duplication risk remains). Consider a more explicit router-mounting convention.
 2. **Very wide schema**: ~140 tables in a single `shared/schema.ts` . Powerful as a single source of truth, but large; could be modularized by domain.
 3. **In-process schedulers**: background jobs run inside the web process, coordinated by DB leases. Works on a single deploy, but doesn't scale horizontally cleanly — a dedicated worker/queue would be more robust.
 4. **Storage layer breadth**: `IStorage` is the chokepoint for all DB access and ownership checks; its size makes it both critical and heavy.
 5. **Feature honesty**: some advertised features were stubs and have been hidden/removed to keep the UI honest (e.g., meal photo analysis). A few flows still depend on external setup to be fully live (Stripe credentials, Resend domain DNS).
 6. **AI cost/latency**: heavy reliance on OpenAI; mix of gpt-4o / gpt-4o-mini and per-day caching mitigates cost, but proactive features and content generation are cost-sensitive surfaces.
 7. **Client size**: ~97 pages / ~111 components in one SPA; lazy loading helps, but bundle size and navigation architecture are worth auditing.
-

14. Quick Facts (for the reviewing AI)

- **Languages**: TypeScript everywhere (React frontend, Node/Express backend, ESM).

- **One server, one port (5000):** Express serves both the API and the Vite/built client.
 - **DB:** PostgreSQL (Neon driver), Drizzle ORM, ~140 tables, 33 SQL migrations, Postgres-backed sessions.
 - **AI:** OpenAI gpt-4o / gpt-4o-mini + TTS Alloy; Perplexity sonar for content.
 - **Payments:** Stripe (monthly/annual subscriptions) via env-var config.
 - **Mobile:** Capacitor (iOS + Android) over the same web build, with native voice assistants.
 - **Auth:** cookie sessions + email/password + Google/Facebook OAuth.
 - **Testing:** Vitest (unit) + Playwright (e2e).
-

End of architecture document.